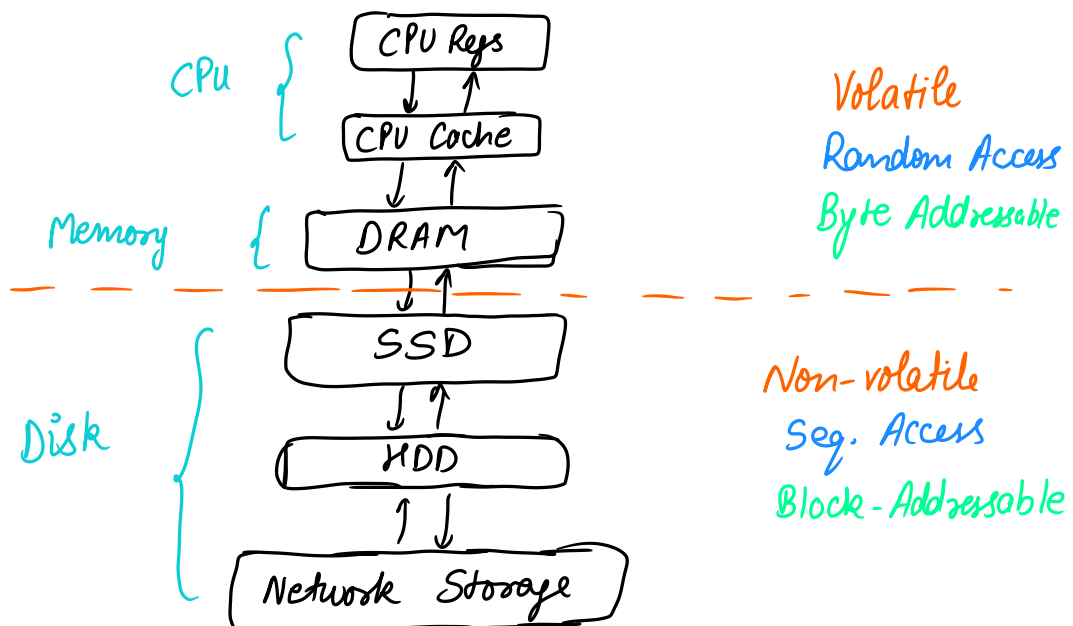


DataBase Storage

→ disk-based architecture

↳ primary storage location of database is on non-volatile disk

↳ dbms's components manage the movement of data b/w non-volatile & volatile storage



→ Access Times:

L1 Cache → 1 ns } CPU | 0.5 ns.
L2 Cache → 4 ns }

DRAM : 100 ns

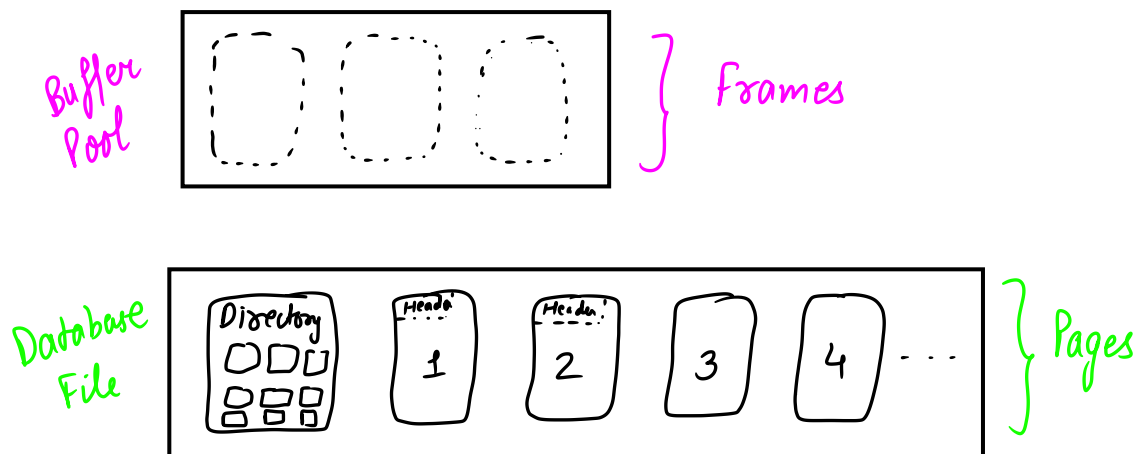
SSD : 16000 ns

HDD : 2000000 ns

Network : 50,000,000 ns
Storage

Tapes : 1000000000 ns

Memory Management & Buffer Pools



⇒ Buffer Replacement Policy

↳ when the DBMS needs to free up a frame to make room for a new page, it must decide which page to evict from buffer pool.

- Correctness
- Accuracy
- Speed
- Meta-data overhead

⇒ Least Recently Used (1965)

↳

↳ maintain a single timestamp of when each page was last accessed. When dbms need to evict a page, select one with oldest timestamp.

#⇒ Clock

↳ approximation of LRU that does not need a separate timestamp.

→ each page has a reference bit.

→ when page is accessed, set its bit to 1.

Organises pages in a circular buffer with a clock hand that sweeps over pages in order.

↳ As hand visits each page, check if bits is set to 1

↳ if yes, set to zero. If no, then evict

Least Frequently Used

↳ maintain an access cnt for each page and then evict page with lowest count.

↳ logarithmic implementation

↳ ignores time & accumulate stale pages with high freq.

LRU-K

→ track history of last K accesses to each page as timestamps and compute the interval access

↳ use this history to estimate the next time pg is going to be accessed.

②? → Maintain in-memory "ghost-cache" for recently evicted pages to prevent them from always being evicted.

ARC : Adaptive Replacement Cache

↳ only implemented in IBM DB2, PostgreSQL

- supports both LRU & LFU by maintaining two lists and then adjusts the size of them based on access pattern.
- maintain ghost lists to remember recent evictions and adapt quickly.

ARC: ADAPTIVE REPLACEMENT CACHE (2003)

Recent List (**T1**):

→ Holds pages that have been accessed once recently.

Recent Ghost List (**B1**):

→ History of pages recently evicted from **T1** (i.e., recency misses).

Frequent List (**T2**):

→ Holds pages that have been accessed at least twice.

Frequent Ghost List (**B2**):

→ History of pages recently evicted from **T2** (i.e., frequency misses).

Target Size Parameter (**p**):

→ Adaptively adjusts how much to favor recency (**T1**) vs. frequency (**T2**).

ARC: LOOKUP PROTOCOL

Cache Miss, Page Found in **B1** (Ghost of **T1**):

→ Increase target size **p** (favor more recency pages).

Cache Miss, Page Found in **B1** (Ghost of **T1**):

- Increase target size **p** (favor more recency pages).
- Move page into **T2** (since it's now accessed again).

Cache Miss, Page Found in **B2** (Ghost of **T2**):

- Decrease target size **p** (favor more frequency pages).
- Move page into **T2**.

Cache Miss, Page Not in Cache or Ghost Lists:

- If **T1** + **B1** is full, evict from **B1** or **T1**.
- If **T2** + **B2** is full, evict from **B2** or **T2**.
- Insert new page into **T1**.

SYSTEMS (FALL 2025)

⇒ buffer flushing.

⇒ Multiple Buffer Pools

- ↳ multiple buffer pool instances
- ↳ per database buffer pool
- ↳ per page type buffer pool

type-specific pools

partitioning memory across multiple pools helps reduce latch contention and improve locality.

Approach 1: Object Id

→ embed an object id in record ids and then maintain a mapping from objects to specific pools

Approach 2: Hashing

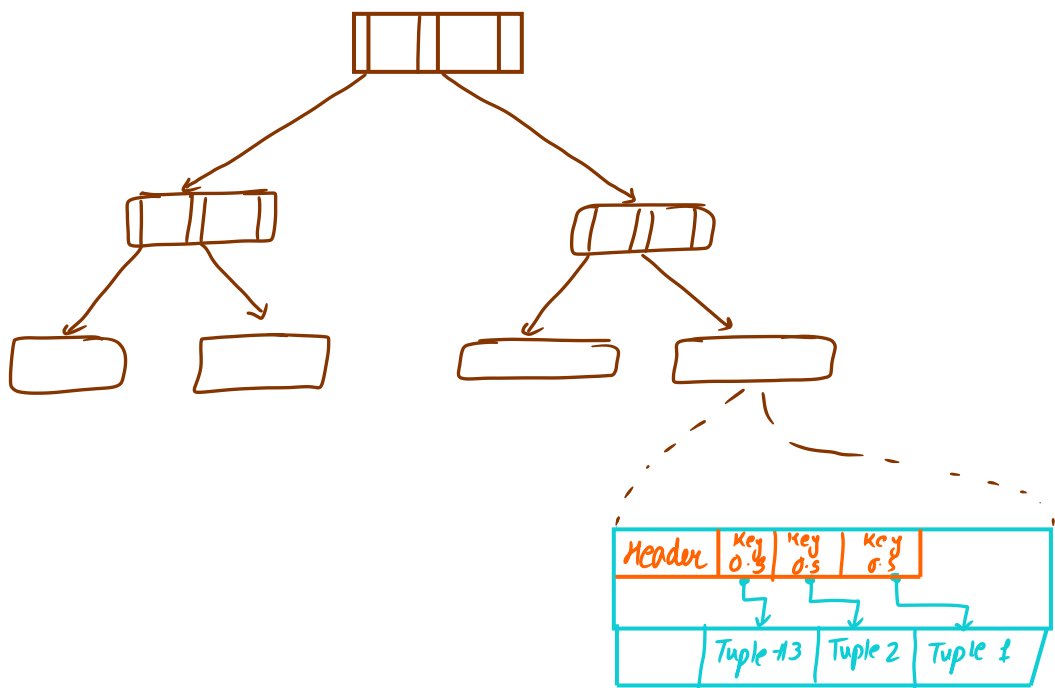
↳ hash the pg id to select which buffer pool to access.

Page Layout

↳ for any page storage architecture, we now need to decide how to organise data inside of a page.

Index-Organised Storage:

DBMS stores a table's tuples as the value of an index data structure



Log-Structured Storage

↳ instead of storing tuples in pages and updating them in place, the DBMS maintains a log that records changes to tuples.

→ each log entry represents a PUT/DELETE operation

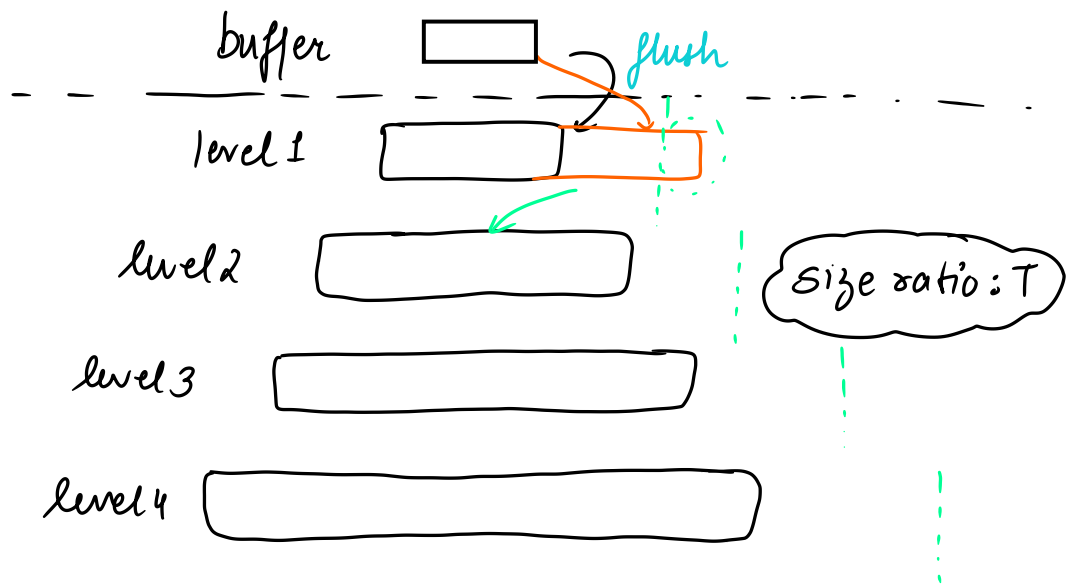
→ originally proposed as log-structured merge trees.

The DBMS applies changes to an in-memory data structure memtable and then writes out changes sequentially to disk as sorted-string (SSTables)

LSM Trees

↳ key-value pairs

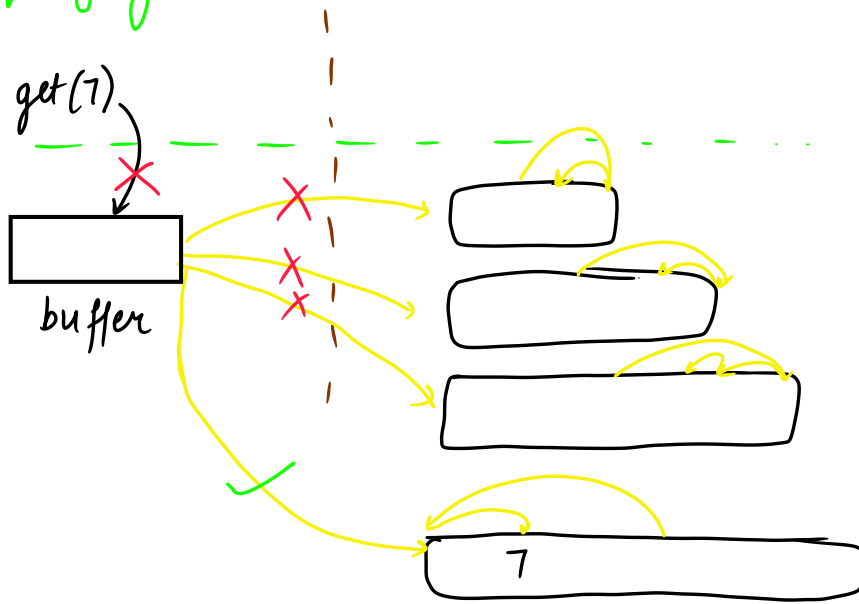
- low ingestion cost
- fast temporal



↳ periodic compaction → adds write amplification

⇒ querying

⇒ querying



↳ becomes logarithmic

$$\hookrightarrow O\left(\sum_i \log_2(P \cdot T^i)\right)$$

⇓

can be avoid by using fence pointers.

$$\hookrightarrow O(\log_T N / (P \cdot B))$$

↳ these unnecessary I/O can be avoided how??

↳ by using bloom filters

↳ false positives:

$$\phi = e^{-\frac{m \cdot BF}{N} \cdot \ln 2^2}$$

Optimising Ingestion in LSM

B⁺ Trees

↳ self-balancing, ordered *m*-way tree for searches, sequential access, insertions & deletion in $O(\log_m n)$ $m \rightarrow$ fanout

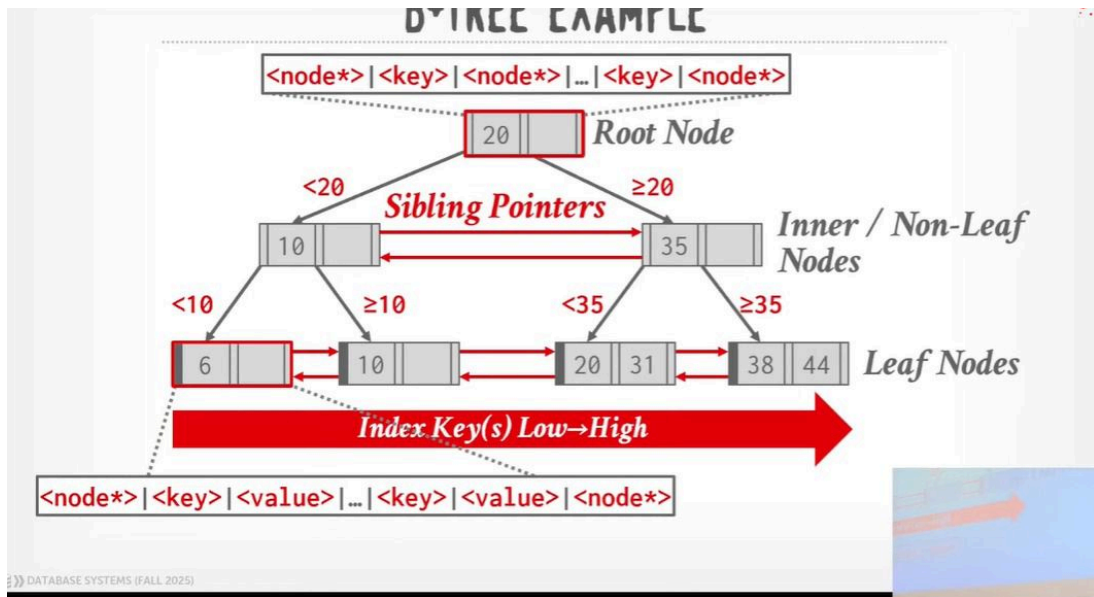
↳ every node except root is half-full

$$m/2 - 1 \leq \# \text{keys} \leq m - 1$$

↳ every inner node with '*k*' keys has *k*+1 non null children

B+TREE EXAMPLE

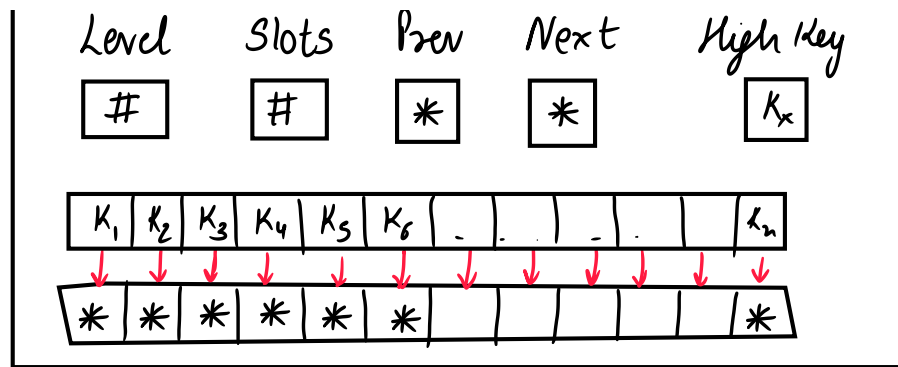
<node*> | <key> | <node*> | ... | <key> | <node*>



↳ comprised of an array of key/value pairs.

⇒ Leaf Node

Level	Slots	Prev	Next	High Key
<u> </u>	<u> </u>	<u> </u>	<u> </u>	<u> </u>



→ Insert

- ↳ find correct leaf node L
- ↳ if L has enough space, done!
- ↳ else
 - split L keys into L and new node L_2
 - ↳ redistribute entries evenly, copy up middle key
 - ↳ insert index entry pointing to L_2 into parent of L .
 - ↳ do this recursively

⇒ Delete

- ↳ find leaf L where entry belongs
- remove
- ↳ if L half-full, done!
- if L has only

if L has only $m/2 - 1$ entries,

- ↳ try to redistribute, borrowing from sibling
- ↳ if re-distribution fails, merge.

Durability

- ↳ The DBMS needs to ensure the following
 - The changes for any txn are durable once the DBMS has told somebody it committed

DBMS has told somebody it committed

→ No partial changes are durable if txn aborted

⇒ UNDO & REDO

↓
The process of removing the effects of an incomplete or aborted txn.

→ The process of re-applying the effects of a committed txn for durability

⇒ Steal Policy

↳ whether the dbms can evict a dirty page buffer pool modified by an uncommitted txn and overwrite the most recent committed version of that object in non-volatile storage.

STEAL: eviction + overwriting is allowed

NO-STEAL: eviction + overwriting is not allowed

⇒ Force Policy

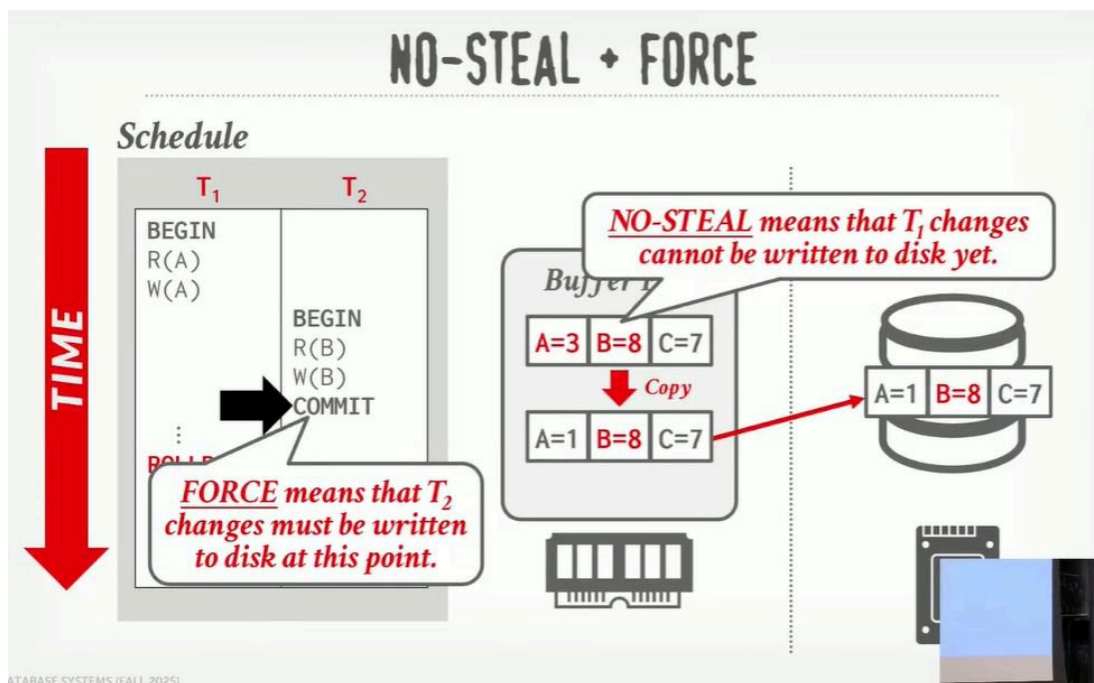
↳ whether the dbms requires that all updates made by a txn are written back to a non-volatile storage before the txn can

storage before the txn can commit.

FORCE: write-back required

NO-Force: Write-back **not** required

eg No-Steal + Force ??



yaad aa jaye to kbhi yaad kr lena
ki yaad
tumhare hoze kahi yaad krne wale
humara ek lauta humse mat chheen lena

Shadow Paging

↳ the system maintains two version of database.

- MASTER: contains only changes from committed txns
- Shadow: temp db with changes made from uncommitted txns.

↳ dbms makes a copy of page before a txns modifies it.

modifies it.

To install updates when a txn commits, overwrite the root so it points to shadow, thereby swapping master & shadow.

buffer pool policy : no-steal + force.

Write-Ahead Logging (WAL)

↳ Maintain a log file separate from data files that contain the changes that txns make to the database.

- Assume log is on stable storage
- Log contains enough information to perform the necessary undo and redo actions

log records enough information to perform the necessary undo and redo actions

↳ dbms must write to the log file records that correspond to changes made to a database object before it can flush that object to the disk.

Buffer Pool Policy: Steal + No force

WAL Protocol

↳ stages all transaction in a volatile storage (usually backed by a buffer pool)

→ All log records pertaining to an updated page are written to a non-volatile storage before the page itself is over-written in non-volatile storage

page flush is over written in non-volatile storage.

→ A txn is not considered committed until all its log records have been written to stable storage.

① Write a **<BEGIN>** record to the log for each txn to mark its starting point.

② Append a record every time a txn changes an object

→ Txn Id

→ Object Id

→ Before Value (UNDO)

→ After Value (REDO)

③ When a txn finishes, the DBMS appends a record to the log

↳ make sure all log records are flushed before it returns acknowledgement to application.

⇒ Implementation

↳ flushing the log buffer to disk every time a txn commits will become a bottleneck.

↳ The DBMS can use a group-commit optimization to batch multiple log flushes together to amortize overhead.

↳ flush when buffer is full

↳ or if there is a timeout.

⇒ Logging Schemes

① Physical

↳ record the byte level change made to specific page.

② Logical

↳ record the high-level operations executed by txns.

③ Physiological

↳ physical to a page, logical within a page

physical to a page, logical within a page
→ hybrid approach will byte-level changes for
a single tuple identified by page id + slot
number
→ no organisation of the page

Checkpoints

- ↳ pause all queries
- ↳ flush all WAL records in memory to disk
- ↳ flush all modified pages in buffer pool to disk
- ↳ write a **<Checkpoint>** entry to WAL and flush to disk
- ↳ resume queries

→ we use the **<checkpoint>** record as the starting point for analyzing WAL

point for analyzing WAL.

CHECKPOINTS

Use the **<CHECKPOINT>** record as the starting point for analyzing the WAL.

Any txn that committed before the checkpoint is ignored (**T₁**).

T₂ + **T₃** did not commit before the last checkpoint.

- Need to redo **T₂** because it committed after checkpoint.
- Need to undo **T₃** because it did not commit before the crash.

WAL

<T1 BEGIN>

<T1, A, 1, 2>

<T1 COMMIT>

<T2 BEGIN>

<T2, A, 2, 3>

<T3 BEGIN>

<CHECKPOINT>

<T2, B, 4, 5>

<T2 COMMIT>

<T3, A, 3, 4>

⋮

💀

DATABASE SYSTEMS (FALL 2025)

ARIES

↳ Algorithms for Recovery and Isolation
Exploiting Semantics

★ Main Ideas

★ Main Ideas

- ↳ use Write Ahead Logging
- ↳ on DBMS restart, retrace actions and restore database to exact state b4 crash.
- ↳ Record undo actions to log to ensure action is not repeated in event of repeated failure.

⇒ WAL Records

- ↳ every log record includes a globally unique log sequence number (LSN)
- ↳ various components in the system keep track of LSN that pertain to them.

⇒ WAL Bookkeeping

↳ each data page contains a **page-LSN**.
→ the LSN of the most recently log record that updated the page.

→ System keeps track of flushed LSN

before the page_x is written,

$$\text{pageLSN}_x \leq \text{flushedLSN}$$

LOG SEQUENCE NUMBERS

Name	Location	Definition
flushedLSN	Memory	Last LSN in log on disk
pageLSN	page _x	Newest update to page _x
recLSN	DPT♦	Oldest update to page _x since it was last flushed
lastLSN	ATT♣	Latest record of txn T _i
MasterRecord	Disk	LSN of latest checkpoint

♦ DPT = Dirty Page Table.

♣ ATT = Active Transaction Table

⇒ Transaction Commit

↳ when transaction commits, the DBMS writes a **commit** record to log & guarantees that all records upto the txn's **COMMIT** record are flushed.

→ when commit succeeds, write **Txn-END** record to log.

⇒ Transaction Abort

↳ aborting a transaction is a special case of ARIES undo operation to only one txn

We need to add another field to our log records

→ **prevLSN**: The previous LSN for this txn.

A txn does not release its locks until it has successfully reverted all of its changes.

⇒ Compensation Log Record

↳ CLR describes the actions taken to reverse the changes of a previous update log record.

→ each CLR contains all the fields of an update record + undo NextLSN pointer.

↳ DBMS adds CLR's to in-memory WAL buffer but it does not wait for them to be flushed before notifying the app. txn's aborted.

⇒ Abort Algorithm

→ first write an abort record to log for the txn

→ analyze the txn's updates in reverse orders.

For each update record:

- Write a CLR entry to the log
- Restore the old value.

→ Lastly write a txn-end record and release blocks.

Fuzzy Checkpoint

→ slightly better checkpoints

↳ pause modifying txns while the dbms takes the checkpoint:

- flushes dirty pages to the disk
- block queries from acquiring write latch on pages
- do not wait until txn's finish before taking the checkpoint.

→ Fuzzy Checkpoint

↳ a fuzzy check point is where the dbms allows active txns to continue to run while the system writes the log record for check point.

New log records to track checkpoint boundaries

↳ Checkpoint-Begin

Checkpoint-End

ARIES

Phase I: Analysis

↳ examine the WAL in forward direction starting at Master Record to identify dirty pages in the buffer pool and active txns at time of the crash

identify dirty pages in one
buffer pool and active txns at time of the crash.

Phase 2: Redo

↳ Repeat all actions starting from an app.
point in the log

Phase 3: Undo

→ reverse the actions of txns that did not commit
before the crash.